

Tratamento de Exceptions em Java

Prof. Hiago

UEPB

O problema: dados inválidos passando despercebidos

```
1 public boolean cadastrarProduto(String nome, double preco,
2     int quantidade) {
3
4     Produto produto = new Produto(nome, preco, quantidade);
5     produtos.add(produto);
6     return true;
7 }
```

Cuidado

Nada impede um preço negativo ou uma quantidade inválida de virar um Produto. O objeto passa a existir "quebrado", e o erro só aparece bem mais tarde, quando alguém tentar calcular o valor total do estoque.

O que é uma Exception?

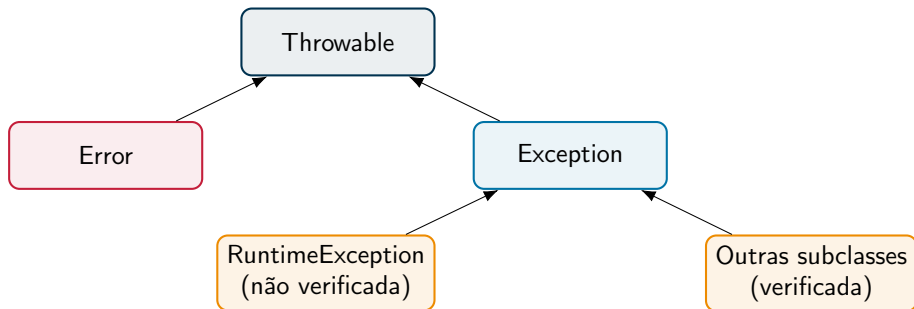
Definição

Uma **exception** é um objeto que representa um evento anormal ocorrido durante a execução do programa. Ela interrompe o fluxo normal até que alguém trate o problema.

Por que usar exceptions em vez de apenas `if` e `return false`?

- Garante que o objeto nunca exista em estado inválido.
- Separa "detectar o erro" de "decidir o que fazer com ele".
- Traz uma mensagem clara sobre o que deu errado.

Hierarquia básica



Dica

Nesta aula, vamos criar uma exception **verificada** (checked) — o compilador obriga quem chama o código a tratar o erro com `try/catch` ou repassá-lo com `throws`.

Sintaxe básica: try e catch

```
1 try {  
2     int resultado = 10 / 0;  
3     System.out.println(resultado);  
4  
5 } catch (ArithmeticException erro) {  
6     System.out.println("Erro: " + erro.getMessage());  
7 }
```

- O bloco try contém o código que pode falhar.
- Se uma exception ocorrer, o restante do try é interrompido e o controle vai direto para o catch.
- Se nada falhar, o catch nunca executa.

throw: lançando uma exception

```
1 public class Produto {  
2     private String nome;  
3     private double preco;  
4     private int quantidade;  
5  
6     public Produto(String nome, double preco, int quantidade)  
7         throws Exception {  
8  
9         if (preco <= 0) {  
10             throw new Exception("Preço inválido.");  
11         }  
12         this.nome = nome;  
13         this.preco = preco;  
14         this.quantidade = quantidade;  
15     }  
16 }
```

throw x throws

throw lança a exception no corpo do método.

throws aparece na assinatura do método e avisa que ele pode lançar aquele tipo de exception.

Você pode criar suas próprias exceptions!

Definição

Em vez de usar a `Exception` genérica, podemos criar uma classe só nossa. O nome da classe já explica o erro, e isso deixa o código bem mais claro para quem lê.

- `Exception` genérica: não diz muito sobre o problema.
- `DadosInvalidosException`: o nome já entrega o que aconteceu.
- A partir de agora, o `Produto` vai lançar a nossa própria exception.

Criando uma exception personalizada

```
1 public class DadosInvalidosException extends Exception {  
2  
3     public DadosInvalidosException(String mensagem) {  
4         super(mensagem);  
5     }  
6 }
```

- Estende Exception → exception verificada (checked).
- O construtor apenas repassa a mensagem para a superclasse.
- Uma exception com nome próprio deixa o erro autoexplicativo.

Regra

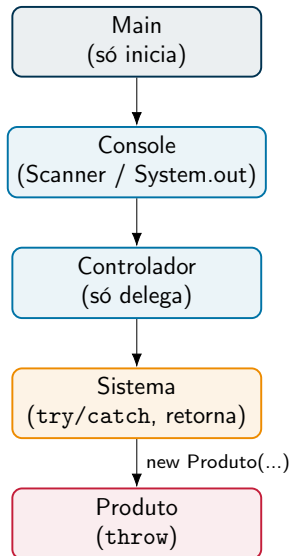
O construtor garante que, se o objeto Produto existir, ele esteja sempre em um estado válido. Se algum dado estiver errado, o objeto simplesmente não chega a existir.

- Nome não pode ser vazio.
- Preço tem que ser maior que zero.
- Quantidade não pode ser negativa.

Código: Produto valida no construtor

```
1 public class Produto {
2     private String nome;
3     private double preco;
4     private int quantidade;
5
6     public Produto(String nome, double preco, int quantidade)
7         throws DadosInvalidosException {
8
9         if (nome == null || nome.trim().isEmpty()) {
10             throw new DadosInvalidosException("Nome inválido.");
11         }
12         if (preco <= 0) {
13             throw new DadosInvalidosException("Preço inválido.");
14         }
15         if (quantidade < 0) {
16             throw new DadosInvalidosException("Quantidade inválida.");
17         }
18         this.nome = nome;
19         this.preco = preco;
20         this.quantidade = quantidade;
21     }
22
23     public String getNome() { return nome; }
24     public double getPreco() { return preco; }
25     public int getQuantidade() { return quantidade; }
26 }
```

Arquitetura do fluxo



Cada camada, uma responsabilidade

- **Main**: só cria o Console e começa o programa.
- **Console**: única classe com Scanner e System.out — lê os dados e mostra os resultados.
- **Controlador**: só delega para o Sistema, sem lógica e sem try/catch.
- **Sistema**: guarda os produtos, faz o try/catch e **retorna** o resultado — nunca imprime nada.
- **Produto**: valida os próprios dados no construtor.

O Sistema nunca imprime nada

Errado

```
1 } catch (DadosInvalidosException erro) {  
2     System.out.println("Erro: " + erro.getMessage());  
3     return false;  
4 }
```

Certo

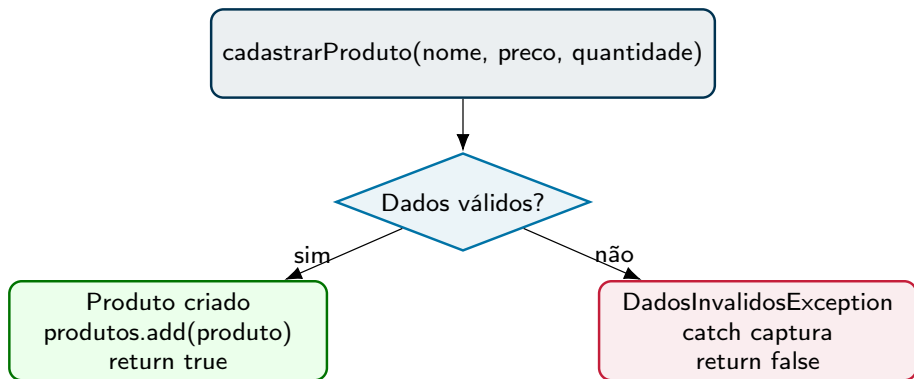
```
1 } catch (DadosInvalidosException erro) {  
2     return false;  
3 }
```

Quem decide **o que mostrar** para quem está usando o programa é o Console, não o Sistema.

Onde tratar? Na classe Sistema

```
1 public class SistemaSupermercado {
2     private ArrayList<Produto> produtos;
3
4     public SistemaSupermercado() {
5         this.produtos = new ArrayList<>();
6     }
7
8     public boolean cadastrarProduto(String nome, double preco,
9         int quantidade) {
10
11         try {
12             Produto produto = new Produto(nome, preco, quantidade);
13             produtos.add(produto);
14             return true;
15
16         } catch (DadosInvalidosException erro) {
17             return false;
18         }
19     }
20
21     public int contarProdutos() {
22         return produtos.size();
23     }
24 }
```

Passo a passo do try/catch



Um método que retorna String

```
1 public String cadastrarProdutoComMensagem(String nome,
2     double preco, int quantidade) {
3
4     try {
5         Produto produto = new Produto(nome, preco, quantidade);
6         produtos.add(produto);
7         return "Produto cadastrado com sucesso.";
8
9     } catch (DadosInvalidosException erro) {
10         return erro.getMessage();
11     }
12 }
```

Dica

Mesmo try/catch, mas agora quem chama recebe uma mensagem pronta em vez de um simples true/false.

Controlador: só delega

```
1 public class ControladorSupermercado {
2     private SistemaSupermercado sistema;
3
4     public ControladorSupermercado() {
5         this.sistema = new SistemaSupermercado();
6     }
7
8     public boolean cadastrarProduto(String nome, double preco,
9         int quantidade) {
10         return sistema.cadastrarProduto(nome, preco, quantidade);
11     }
12
13     public String cadastrarProdutoComMensagem(String nome,
14         double preco, int quantidade) {
15         return sistema.cadastrarProdutoComMensagem(nome, preco,
16             quantidade);
17     }
18 }
```

Dica

O Controlador nunca vê um try/catch nem um System.out. Ele só repassa a chamada.

A classe Console: Scanner e System.out

```
1 public class Console {
2     private Scanner leitor;
3     private ControladorSupermercado controlador;
4
5     public Console() {
6         this.leitor = new Scanner(System.in);
7         this.controlador = new ControladorSupermercado();
8     }
9
10    public void cadastrarProduto() {
11        System.out.print("Nome: ");
12        String nome = leitor.nextLine();
13        System.out.print("Preço: ");
14        double preco = leitor.nextDouble();
15        System.out.print("Quantidade: ");
16        int quantidade = leitor.nextInt();
17
18        boolean sucesso = controlador.cadastrarProduto(nome,
19            preco, quantidade);
20
21        if (sucesso) {
22            System.out.println("Produto cadastrado com sucesso.");
23        } else {
24            System.out.println("Não foi possível cadastrar.");
25        }
26    }
27 }
```

Console com o método que retorna String

```
1 public void cadastrarProdutoComMensagem() {  
2     System.out.print("Nome: ");  
3     String nome = leitor.nextLine();  
4     System.out.print("Preço: ");  
5     double preco = leitor.nextDouble();  
6     System.out.print("Quantidade: ");  
7     int quantidade = leitor.nextInt();  
8  
9     String resultado = controlador.cadastrarProdutoComMensagem(  
10         nome, preco, quantidade);  
11  
12     System.out.println(resultado);  
13 }
```

Dica

Quando o Sistema já devolve uma mensagem pronta, o Console só precisa exibi-la — não precisa decidir mais nada.

Main: só inicia o Console

```
1 public class Main {  
2     public static void main(String[] args) {  
3         Console console = new Console();  
4         console.cadastrarProduto();  
5     }  
6 }
```

Essa divisão é uma boa prática?

Regra de Ouro das Exceptions

Quem **cria** o objeto decide o que fazer quando a criação falha. Quem **conversa** com o usuário decide o que mostrar.

- O **Construtor** só sabe validar os próprios dados.
- O **Sistema** chama `new Produto(...)`, faz o `try/catch` e retorna o resultado — nunca imprime.
- O **Console** é o único lugar que decide o que aparece na tela.
- O **Controlador** continua com uma única responsabilidade: delegar.

Cuidado

- Colocar `System.out` dentro do Sistema — quem decide o que mostrar é o Console.
- Colocar o `try/catch` no Controlador — mistura delegação com tratamento de erro.
- Fazer `catch (Exception e) { }` vazio — o erro desaparece silenciosamente.
- Validar os dados fora do construtor e só depois criar o objeto — o Produto deixa de garantir sua própria validade.

Exemplo

Crie a exception `EstoqueInsuficienteException` e lance-a em um método `venderProduto(String nome, int quantidade)` sempre que a quantidade pedida for maior que a quantidade em estoque. Trate essa exception na classe `Sistema` (sem imprimir nada) e depois crie um método no `Console` que chama o `Controlador` e mostra o resultado na tela.

Checklist

- O construtor lança a exception quando os dados são inválidos?
- Quem chama `new` (o Sistema) é quem faz o `try/catch`?
- O Sistema só retorna valores, sem nenhum `System.out`?
- Só o Console usa `Scanner` e `System.out`?
- O Controlador continua sem saber que a exception existe?

Dúvidas?